

A Neurosymbolic Framework for Interpretable Cognitive Attack Detection in Augmented Reality

Rongqian Chen¹ Allison Andreyev¹ Yanming Xiu² Joshua Chilukuri² Shunav Sen²
 Mahdi Imani⁴ Bin Li³ Maria Gorlatova² Gang Tan³ Tian Lan¹

¹George Washington University ²Duke University
³Pennsylvania State University ⁴Northeastern University

Abstract

Augmented Reality (AR) enriches human perception by overlaying virtual elements onto the physical world. However, this tight coupling between virtual and real content makes AR vulnerable to cognitive attacks: manipulations that distort users' semantic understanding of the environment. Existing detection methods largely focus on visual inconsistencies at the pixel or image level, offering limited semantic reasoning or interpretability. To address these limitations, we introduce CADAR, a neuro-symbolic framework for cognitive attack detection in AR that integrates neural and symbolic reasoning. CADAR fuses multimodal vision-language representations from pre-trained models into a perception graph that captures objects, relations, and temporal contextual salience. Building on this structure, a particle-filter-based statistical reasoning module infers anomalies in semantic dynamics to reveal cognitive attacks. This combination provides both the adaptability of modern vision-language models and the interpretability of probabilistic symbolic reasoning. Preliminary experiments on an AR cognitive-attack dataset demonstrate consistent advantages over existing approaches, highlighting the potential of neuro-symbolic methods for robust and interpretable AR security.

1. Introduction

Augmented Reality (AR) and Mixed Reality (MR) technologies increasingly overlay virtual content onto the physical world, reshaping how users perceive and interact with their surroundings [33]. This tight integration creates new vulnerabilities, as users tend to trust overlaid visual and spatial cues. Adversaries can exploit this trust to launch cognitive attacks, where malicious virtual content is injected or altered to mislead, distract, or manipulate users without their awareness [8, 24, 28, 29]. Unlike traditional cybersecurity threats that target system integrity, cognitive attacks

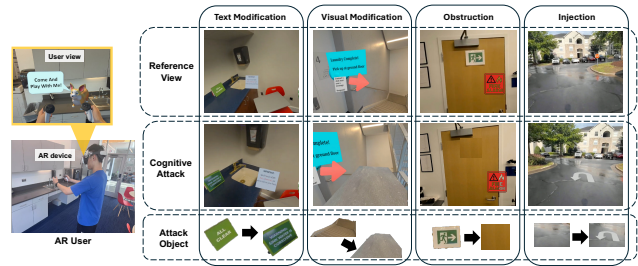


Figure 1. Illustrative AR/MR cognitive attacks. Top row: original scenes; bottom row: manipulated scenes. (a) Text Modification Attack. (b) Visual Modification attack. (c) Obstruction Attack. (d) Injection Attack.

operate at the perceptual level, subtly manipulating users' semantic understanding of a scene (examples in Fig.1).

Prior work on detecting anomaly and attacks often employs computer vision techniques to identify visual changes or adversarial distortions [1, 11, 12, 15, 19, 20, 26]. However, these methods rely on pixel/image processing, lacking the ability to model the human perception process that not only involves prior knowledge and contextual information, but also selectively focuses on aspects of the visual field for semantical interpretation. As a result, these methods may struggle to capture smart cognitive attacks manipulating only a small number of pixels and also may suffer from false positives due to benign visual perturbations that do not affect users' semantic interpretation [15]. More recent studies propose pre-trained vision-language models (VLMs) [22, 30] can achieve semantic-level detection by processing visual features and incorporating human-like knowledge. But the reasoning of pre-trained VLMs is a black-box approach offering limited interpretability. It is difficult to leverage domain-specific knowledge or model scene/context evolutions or temporal correlations in cognitive attack detection.

In this paper, we consider cognitive attacks that aim to manipulate users' semantic perception in AR/MR. We propose CADAR (Cognitive Attack Detection in Augmented

Reality), a neuro-symbolic approach encompassing two key components: (i) a symbolic perception-graph model, inspired by knowledge and scene graphs [5, 13, 23, 31]. It is extracted using pre-trained neural VLM models to represent scene/context evolution at the semantic-level, and (ii) a particle-filter model, which is commonly used in non-linear and stochastic systems. It applies a sequential Monte Carlo method to the perception graph for reasoning and detecting cognitive attacks, stabilizes inherently noisy or inconsistent VLM outputs, and mitigates errors or missed detections through probabilistic fusion. It also offers granular, flexible temporal detection, using reference sequences, frame-wise inference localizes attacks per frame, to produce a variable number of labels when needed. Thus, the *CADAR* approach inherits the adaptability of the pre-trained VLM (neural part), as well as the interpretability and reasoning rigor of particle filtering (symbolic part). It semantically interprets and models the sequential scene changes for robust cognitive attack detection in AR/MR, overcoming existing work’s limitations.

We summarize our contributions as follows:

- **Neuro-symbolic framework.** We introduce a hybrid architecture that converts VLM outputs into symbolic perception graphs for explicit and interpretable reasoning in AR cognitive attack detection.
- **Perception graph schema.** We design a spatio-temporal graph representation that integrates multimodal information from image/text encoders and detection modules to capture scene evolution.
- **Particle-filter reasoning.** We apply particle filtering to symbolic perception graphs, improving robustness to noisy VLM outputs and enabling principled statistical anomaly detection.
- **CADAR-50K dataset.** We build the first public AR cognitive attack dataset and demonstrate that *CADAR* achieves 74.6% accuracy, significantly outperforming all baseline methods.

The rest of the paper is organized as follows: Section 2 reviews related work; Section 3 presents our method; Section 4 reports experimental results; and Section 5 concludes the paper.

2. Related Work

Cognitive Attacks in AR/MR. One well-studied category of visual attacks is adversarial attacks, which introduce imperceptible yet malicious perturbations to input data to mislead machine learning models [17, 32]. In contrast, cognitive attacks target perception-level semantics and aim to alter a human user’s understanding of a scene [8, 28]. Augmented Reality (AR) systems, which overlay virtual content onto the physical world to enhance perception and decision-making, are particularly vulnerable to such attacks. However, research on cognitive attacks in AR remains limited.

Existing methods often rely on access to the original, unaltered scene for comparison or employ pre-trained VLMs with limited interpretability.

Neuro-symbolic Methods. Neuro-symbolic approaches couple deep neural networks with symbolic reasoning—i.e., reasoning over explicitly represented knowledge in formal languages, to capitalize on the complementary strengths of subsymbolic pattern recognition and symbolic compositionality [4, 18]. Within knowledge-graph research, most neuro-symbolic work addresses link prediction or graph completion, where the hybrid models offer interpretable rules, guided learning, and more transparent decision pathways [10]. Yet, like standard supervised techniques, many of these solutions still demand sizable labeled datasets. In contrast, our framework introduces a neuro-symbolic reasoning module that combines explicitly encoded rules with statistics, achieving high interpretability while remaining data-efficient.

Particle Filter and Statistical Reasoning. Particle filter is a sequential Monte Carlo algorithm for recursive Bayesian estimation [7], it propagates a set of weighted samples to approximate the posterior state distribution, outperforming classical Kalman filters in strongly non-linear or non-Gaussian settings [6, 14]. They are widely used in surveillance, robotics, and navigation, leveraging visual, geometric, and motion cues such as color, texture, and shape [3]. In *CADAR*, we model graph nodes and edges as stochastic particles, enabling robust reasoning under noisy or inconsistent VLM outputs.

3. Methodology

In this section, we define the four types of cognitive attacks in AR scenes and introduce our neuro-symbolic framework for detecting such attacks. As illustrated in Fig. 2, the framework consists of two main components: the perception graph model and the particle filter-based detection module. The perception graph encodes each frame as a 3D spatiotemporal structure, where nodes represent objects and edges capture their relationships across time (Fig. 3). These structured graphs are then passed into the attack detection algorithm (Fig. 4), which reasons over the particle filter to identify cognitive attacks.

3.1. Cognitive Attacks

We investigate four representative cognitive attack types that deliberately compromise the semantic integrity of AR scenes (Fig. 1). The text-modification attack manipulates on-scene text—such as changing “NO PARKING” to “FREE PARKING”—to reverse intended meanings or inject false instructions, thereby deceiving both human users and perception systems. The visual-modification attack distorts object appearance or placement—for example, turning a green

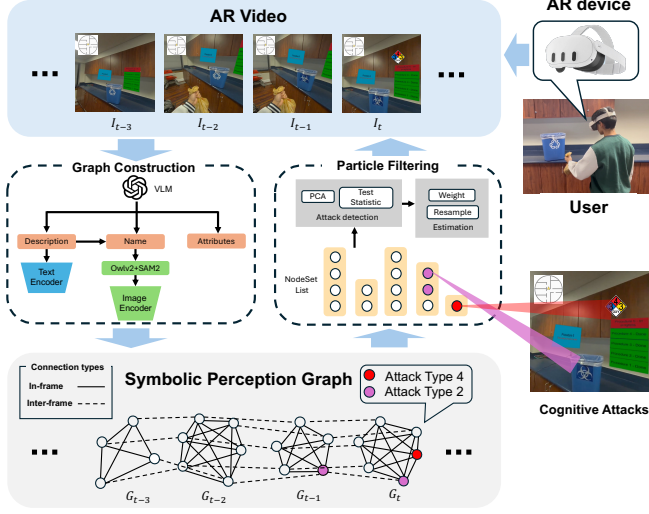


Figure 2. A system overview of CADAR. Sequential video frames are first transformed into spatial-temporal symbolic perception graphs. The attack-detection module then analyzes these graphs to identify, classify, and localize adversarial attacks, such as the visual modification and injection attacks shown in the examples.

light to red or relocating a stop sign—leading to recognition errors and misinformed user decisions. The obstruction attack targets critical information by occluding or deleting essential cues like exit signs, disrupting safety awareness and expected graph relations. Finally, the injection attack introduces fictitious elements, such as fake hazard symbols or virtual labels, that embed misleading cues, divert user attention, and corrupt subsequent reasoning processes.

3.2. Perception Graph Model

The perception graph model is a symbolic representation of perception, simulating how humans perceive, interpret, and prioritize information. It collects, structures, and stores scene data that enables formal reasoning later. The graph leverages pre-trained VLMs to approximate human interpretation of sensory input. Given AR imagery and task context, the model fuses multimodal signals into a compact graph and embeds their semantics in a latent perceptual space.

A perception graph consists of nodes and edges. Objects are represented as nodes, and object relationships are represented as edges. Given the context information and prompt (shown in the Fig.3), the VLM would extract the related objects with their relationships in the scene. At timestep t , the Perception Graph G_t is generated as follows.

3.2.1. Graph Construction.

At discrete timestep $t \in \{1, \dots, T\}$ we ingest

$$I_t \in \mathbb{R}^{H \times W \times 3}, \quad c_t \in \mathcal{C}, \quad (1)$$

where I_t is the AR video frame and c_t encodes task context.

A pre-trained vision-language module $\mathcal{F}_{\text{VLM}}$ lifts the frame into a spatio-temporal perception graph

$$G_t = (\mathcal{V}_t, \mathcal{E}_t) = \mathcal{F}_{\text{VLM}}(I_t, c_t), \quad (2)$$

with nodes $\mathcal{V}_t$ and edges $\mathcal{E}_t$:

$$\mathcal{V}_t = \{v_i^t\}_{i=1}^{N_t}, \quad \mathcal{E}_t = \{e_{ij}^t \mid v_i^t, v_j^t \in \mathcal{V}_t\}. \quad (3)$$

For reasoning that needs the *entire* observation history (e.g. long-range causal inference) we define GraphSet

$$\hat{G}_t = (\mathcal{V}_{1:t}, \mathcal{E}_{1:t}), \quad \mathcal{V}_{1:t} = \bigcup_{s=1}^t \mathcal{V}_s, \quad \mathcal{E}_{1:t} = \bigcup_{s=1}^t \mathcal{E}_s. \quad (4)$$

Thus, the GraphSet $\hat{G}_t$ is the union of all spatial and temporal connections that stitch object instances across time steps 1:t. Each node still carries its original timestamp to preserve the temporal ordering.

Each node in the perception graph is represented by several attributes, including its ID, attack type, and timestep, along with a semantic description, visual features such as segmentation masks, a recognition probability score, contextual importance based on task relevance, and a reasonability score ranging from -3 to $+3$. These attributes are encoded using pre-trained vision-language and segmentation models to capture multimodal object representations. Each edge connects two nodes and includes similar parameters (node IDs, attack type, and timestep), along with a textual description capturing semantic, functional, navigational, or instructional relationships. Edge importance is derived from the contextual salience of the connected nodes.

3.2.2. NodeSet and EdgeSet.

After defining the temporal graph structure $\hat{G}_t$, which links perception graphs across time, we introduce the NodeSet and EdgeSet to represent the temporally connected nodes and edges, respectively.

At each time step t , the NodeSet for object i is denoted as $\mathcal{N}_{i,t}$, and the collection of all such NodeSets is represented as the NodeSet List $\mathcal{N}_t$.

$$\mathcal{N}_{i,t} = \{v_i^k \mid 1 \leq k \leq t\}, \quad i \in \mathcal{I}_t, \quad (5)$$

$$\mathcal{N}_t = \{\mathcal{N}_{i,t} \mid i \in \mathcal{I}_t\}.$$

where $\mathcal{I}_t = \bigcup_{s=1}^t \{i \mid v_i^s \in \mathcal{V}_s\}$ is the set of all object IDs observed so far.

Similarly, EdgeSet and EdgeSet List is defined as:

$$\mathcal{E}_{i,j,t} = \{e_{ij}^k \mid 1 \leq k \leq t\}, \quad i, j \in \mathcal{I}_t, \quad (6)$$

$$\mathcal{E}_t = \{\mathcal{E}_{i,j,t} \mid i, j \in \mathcal{I}_t\}.$$

Thus the GraphSet $\hat{G}_t$ can also be represented by NodeSet and EdgeSet:

$$\hat{G}_t = (\mathcal{V}_{1:t}, \mathcal{E}_{1:t}), \quad \mathcal{V}_{1:t} = \bigcup_{i \in \mathcal{I}_t} \mathcal{N}_{i,t}, \quad \mathcal{E}_{1:t} = \bigcup_{i,j \in \mathcal{I}_t} \mathcal{E}_{i,j,t}. \quad (7)$$

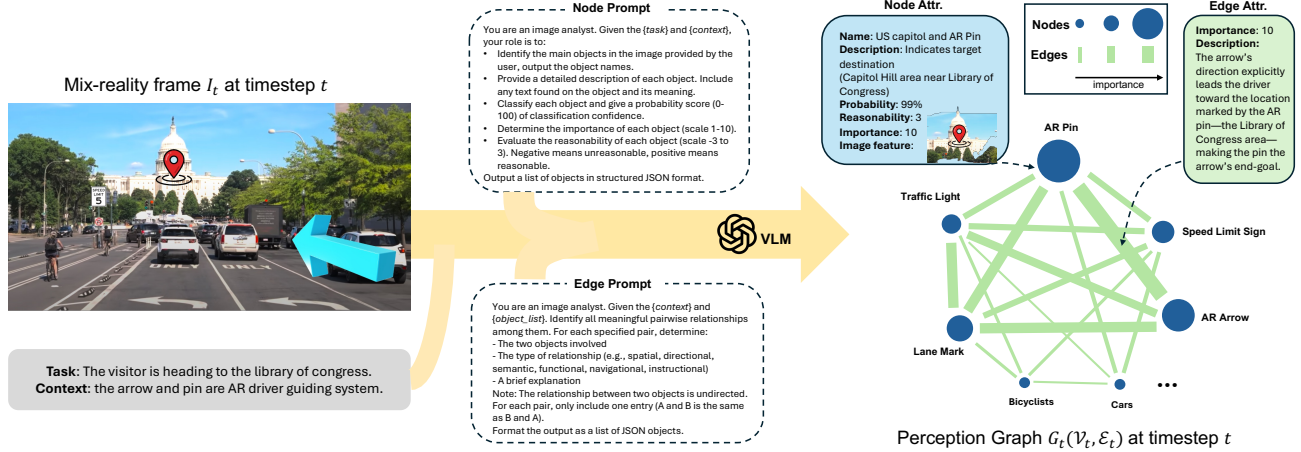


Figure 3. Symbolic perception graph generation: given a video frame and its contextual description, VLMs generate the corresponding perception graph at time step t .

3.3. Particle Filter Model

Our particle filter model tracks graph evolution and analyzes adversarial perturbations. It proceeds in three stages: prediction and matching, attack detection, and state estimation (Fig. 4). Each entity (node or edge) is treated as a particle. At each timestep, particles match the incoming observations with prior particles. The attack detection algorithm applies statistical tests to newly matched particles and discards those marked as attacked. Finally, weighting, resampling, and estimation produce an updated, temporally consistent graph state, preserving persistent NodeSet/EdgeSet identities and the relational structure over time.

3.3.1. Prediction and Matching.

To maintain persistent object identities across frames, we equip the NodeSet with the Prediction-Matching algorithm and use it to decide whether a newly detected Node belongs to an existing object list or creates a new one.

For any node v_i^t we denote $\mathbf{n}(v) = \text{name embedding}$, $\mathbf{d}(v) = \text{description embedding}$, $\mathbf{f}(v) = \text{visual feature embedding}$, all $\in \mathbb{R}^{d_m}$. Cosine similarity for modality $m \in \{n, d, f\}$ is defined as $\text{sim}_m(u, v)$.

Let $\mathcal{A}_{t-1}$ be the set of NodeSet IDs that are still active at time $t-1$. Each set maintains an Active Level $A_i^t \in [0, 3]$ (initialized to 3) indicating recent presence: if unmatched at time t , $A_i^t \leftarrow \max(0, A_i^{t-1} - 1)$; if matched, $A_i^t \leftarrow \min(3, A_i^{t-1} + 1)$. When $A_i^t = 0$, the node is considered absent and ceases matching. For every active object ID $i \in \mathcal{A}_{t-1}$ and each modality $m \in \{n, d, f\}$ compute the mean and standard deviation of pairwise similarities inside the NodeSet:

$$\mu_{i,m}^{t-1} = \frac{2}{|\mathcal{N}_{i,t-1}|(|\mathcal{N}_{i,t-1}| - 1)} \sum_{\substack{p < q \\ v_i^p, v_i^q \in \mathcal{N}_{i,t-1}}} \text{sim}_m(v_i^p, v_i^q),$$

$$\sigma_{i,m}^{t-1} = \text{StdDev}\{\text{sim}_m(v_i^p, v_i^q) \mid p < q, v_i^p, v_i^q \in \mathcal{N}_{i,t-1}\}. \quad (8)$$

To match the observed node to NodeSet, given a fresh node v_j^t and a candidate NodeSet i , define

$$s_{j,i,m} = \frac{1}{|\mathcal{N}_{i,t-1}|} \sum_{v \in \mathcal{N}_{i,t-1}} \text{sim}_m(v_j^t, v), \quad m \in \{n, d, f\}. \quad (9)$$

Candidate filter ($\leq 1\text{std}$ in every modality):

$$\mathcal{C}_j = \left\{ i \in \mathcal{A}_{t-1} \mid |s_{j,i,m} - \mu_{i,m}^{t-1}| \leq \sigma_{i,m}^{t-1}, \forall m \in \{n, d, f\} \right\}. \quad (10)$$

Finally, the assignment rule based on measuring the remaining candidate distance:

$$\mu_t(v_{j,t}) = \begin{cases} \arg \min_{i \in \mathcal{C}_j} \sum_{m \in \{n, d, f\}} |s_{j,i,m} - \mu_{i,m}^{t-1}|, & \mathcal{C}_j \neq \emptyset, \\ \text{new_id}, & \mathcal{C}_j = \emptyset. \end{cases} \quad (11)$$

If $\mu_t(v_j^t)$ returns an existing ID, append v_j^t to that NodeSet and update A_i^t ; otherwise, spawn a new NodeSet. Confirmed edges e_{ab}^t are then inserted into $E_{\mu_t(a)\mu_t(b),t}$ as before. After matching, the NodeSet goes into the Attack detection process.

3.3.2. Attack Detection.

To detect text (Type 1) and visual (Type 2) modification attacks, those that manifest as anomalies in embedding space, we employ a dedicated statistical procedure. Because the same description embeddings are later reused when assessing EdgeSets, we illustrate the method with the NodeSet case; the extension to edges is entirely analogous. A NodeSet with a fresh match in frame t is denoted $\mathcal{N}_{i,t} = \mathcal{N}_{i,t-1} \cup \{v_i^t\}$. Let $\tilde{\mathcal{N}}_{i,t-1} \subseteq \mathcal{N}_{i,t-1}$ be the subset of nodes that *have not* been flagged as attacks in earlier frames; these constitute the “clean” reference set.

For each modality $m \in \{d, f\}$ (description embedding, visual feature embedding), we use the Principal Component

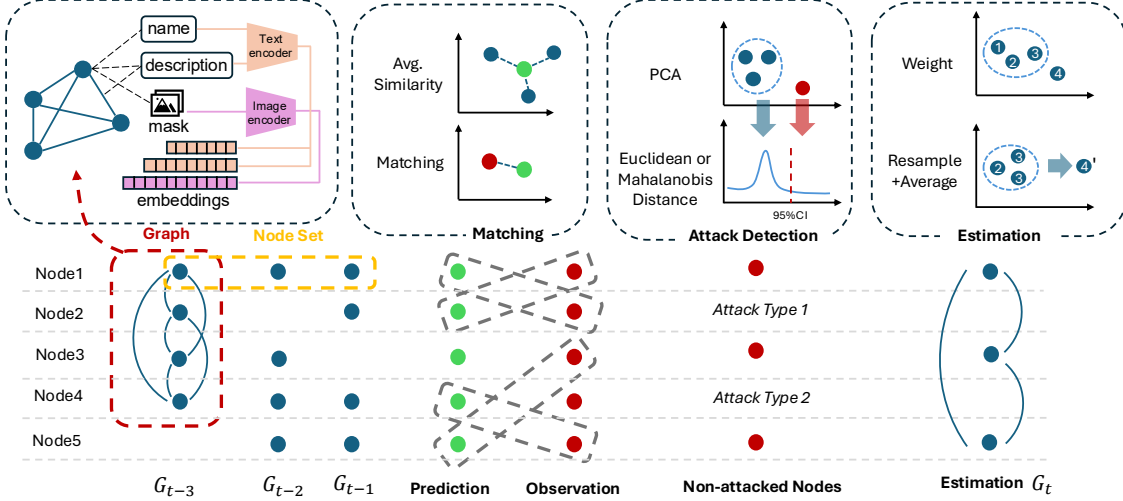


Figure 4. The particle filter-based attack detection framework consists of three main modules: matching, attack detection, and state estimation.

Analysis (PCA) to project it to a low-dimensional space, which removes noisy, low-variance directions. A PCA mapping on the clean reference data is computed by

$$\mathbf{W}_{i,m}^{t-1} = \text{PCA}(\{\mathbf{m}(v)\}_{v \in \tilde{\mathcal{N}}_{i,t-1}}, k_m), \quad (12)$$

where k_m is chosen such that $\geq 99\%$ variance is retained. the resulting embeddings are modeled as Gaussian distributions:

$$\mathbf{z}_m(v) = \mathbf{W}_{i,m}^{t-1 \top} \mathbf{m}(v) \sim \mathcal{N}(\mathbf{c}_{i,m}^{t-1}, \Sigma_{i,m}^{t-1}) \quad (13)$$

with the centroid and covariance of PCA-reduced embeddings

$$\mathbf{c}_{i,m}^{t-1} = \frac{1}{|\tilde{\mathcal{N}}_{i,t-1}|} \sum_{v \in \tilde{\mathcal{N}}_{i,t-1}} \mathbf{z}_m(v), \quad \Sigma_{i,m}^{t-1} = \text{Cov}\{\mathbf{z}_m(v)\}. \quad (14)$$

Use the Mahalanobis distance for measuring Text and Visual Modification attack. For a new node v_i^t compute

$$\mathcal{M}_{i,t}^{(m)} = (\mathbf{z}_m(v_i^t) - \mathbf{c}_{i,m}^{t-1})^\top (\Sigma_{i,m}^{t-1})^{-1} (\mathbf{z}_m(v_i^t) - \mathbf{c}_{i,m}^{t-1}), \quad (15)$$

Assume under the no-attack hypothesis, the squared Mahalanobis statistic $\mathcal{M}_{i,t}^{(\cdot)} \sim \chi_{k,\alpha}^2$, which is the $(1 - \alpha)$ -quantile of a chi-square distribution with k degrees of freedom. So flag the attack as

$$v_i^t \text{ as } \begin{cases} \text{Type 1 Attack(text)} & \text{if } \mathcal{M}_{i,t}^{(d)} > \chi_{k_d,\alpha}^2, \\ \text{Type 2 Attack(image)} & \text{if } \mathcal{M}_{i,t}^{(f)} > \chi_{k_f,\alpha}^2. \end{cases} \quad (16)$$

For the structural attacks, specifically object obstruction (Type 3) and object injection (Type 4), the detector relies on lightweight temporal heuristics rather than statistical distance. After each matching step, the algorithm keeps a two-frame sliding window of NodeSet activity. If an object that

previously carried a high task-importance score ($\pi \geq \pi_{\text{high}}$) fails to obtain a match in both the current and the immediately preceding frame, the system labels the disappearance an obstruction attack. Conversely, any NodeSet that first appears within the last two frames and whose reasonability score falls below a predefined threshold ($\rho \leq \rho_{\text{low}}$) is treated as an injection attack.

In the attack detection final step, if v_i^t is flagged as attack, it is appended to the $\tilde{\mathcal{N}}_{i,t}$ but excluded from future Gaussian updates; otherwise going to the Node Estimation process, including weight and resample, then stored in a clean reference $\tilde{\mathcal{N}}_{i,t}$.

3.3.3. Estimation.

After a new observation $v_{i,t}$ has been matched to object i and detected as a non-attack node, the particle filter estimates the new node by refining that observation by resampling historical detections that resemble it. The estimation proceeds as follows.

Let the newly matched observation of NodeSet i at frame t be $v_{i,t}$ and denote by

$$\tilde{\mathcal{N}}_{i,t-1} = \{v^{(1)}, \dots, v^{(M)}\} \subseteq \tilde{\mathcal{N}}_{i,t-1}, \quad M \geq 5,$$

the set of *non-attack* detections accumulated up to $t-1$. Considering each attribute as a particle, the filter updates $v_{i,t}$ by weighted resampling over this reference set.

For any scalar attribute $a \in \{p, \pi, \rho\}$ (probability, importance, reasonability), let $a^* = a(v_i^t)$ and $a^{(j)} = a(v^{(j)})$ ($j = 1 \dots M$). Define the weight

$$\sigma_a = \max(\text{Std}\{a^{(1:M)}\}, \sigma_{\min}), \quad (17)$$

$$w_j = \frac{\exp[-(a^{(j)} - a^*)^2 / (2\sigma_a^2)]}{\sum_{k=1}^M \exp[-(a^{(k)} - a^*)^2 / (2\sigma_a^2)]}. \quad (18)$$

So w_j is the normalized weight of the particle, based on its distance to the new particle. Similarly, for embedding attributes, modality $m \in \{n, d, f\}$ (name, description, image) let $m^* = m(v_i^t)$ and $m^{(j)} = m(v^{(j)})$. Weight is defined as

$$sim_j = \frac{m^{(j)} \cdot m^*}{\|m^{(j)}\| \|m^*\|}, \quad w_j = \frac{\exp(\lambda_m sim_j)}{\sum_{k=1}^M \exp(\lambda_m sim_k)}, \quad (19)$$

with a scale factor $\lambda_m > 0$.

Then use the weight to resample M indices $s_1, \dots, s_M \sim \text{Multinomial}(w_{1:M})$ yields estimated scalar particle and embedding particle:

$$\hat{a} = \frac{1}{M} \sum_{k=1}^M a^{(s_k)}, \quad \hat{m} = \frac{\sum_{k=1}^M m^{(s_k)}}{\left\| \sum_{k=1}^M m^{(s_k)} \right\|}, \quad (20)$$

where $\hat{a}$ is the estimated new scalar particle.

Finally, replacing each raw attribute in v_i^t by its resampled estimate gives the estimated node

$$\hat{v}_{i,t} = (\hat{p}, \hat{\pi}, \hat{\rho}, \hat{n}, \hat{d}, \hat{f}), \quad (21)$$

which becomes the particle filter’s estimation state for object i at time t . If $M < 5$ (insufficient history), we skip resampling and simply set $\hat{v}_{i,t} = v_{i,t}$.

4. Experiment

In this section, we provide a detailed description of the *CADAR* experimental settings used for performance evaluation. We then present and analyze the numerical results of our experiments, demonstrating that *CADAR* significantly outperforms several baselines. An ablation study is also conducted to analyze the contributions of our key components.

4.1. Experimental Settings

CADAR-50K Dataset. The *CADAR*-50K dataset extends the AR-VIM dataset [27], which was designed to evaluate visual information manipulation in AR environments. *CADAR*-50K is the first publicly available dataset for AR cognitive-attack analysis, containing 375 videos across diverse scenes and more than 50K frames, covering all seven categories (Fig. 5 and Fig. 6). Each sample is an 8–12 second AR-captured video (recorded using a Meta Quest 3 or Samsung Galaxy S24), where virtual content is introduced at a specific frame to induce an attack. For the non-attack elements, we use irrelevant background objects, benign AR elements, and semantically benign prompts that can interfere with VLM outputs but do not alter the human-perceived task semantics.

Considering scene availability, the dataset was collected using two complementary methods: (1) a virtual-scene method, where fully synthetic environments are first generated and displayed on a monitor or VR headset, and AR

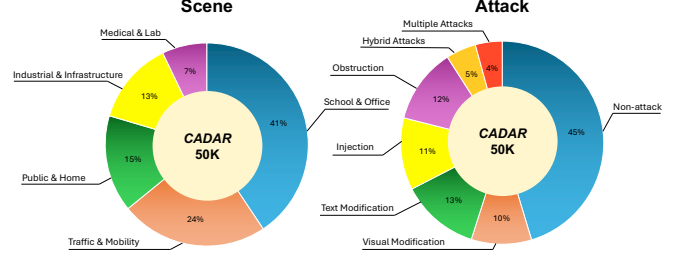


Figure 5. *CADAR*-50K dataset overview.

content is then overlaid onto these virtual scenes before being captured through video recording. This enables rapid, controllable creation of diverse attack scenarios; and (2) a real-world method, where AR headsets record first-person views in physical environments, introducing natural noise, lighting variation, and depth cues.

Task, Baselines and Metrics. We evaluate our model on the task of video-based cognitive attack classification. The *CADAR* dataset supports frame-wise detection, variable-length multi-label outputs, and localization of simultaneous attacks. We compare *CADAR* against two categories of baselines that similarly accept multiple reference frames as input and output attack classes: (1) multimodal VLMs, including state-of-the-art *OpenAI GPT-5*, *Google Gemini-2.5* [9] and *xAI Grok-4*, which perform fine-grained frame-level detection; and (2) supervised video models, the CNN-based *3D-ResNet* [25] and transformer-based *ViViT* [2], which output a single prediction per video.

In our first experiment (Table 1), we use class-specific accuracy and F1-score to assess overall detection performance. The class-specific accuracy is defined as $TP/(TP + FN + FP)$, which penalizes both missed detections and false alarms for that class while ignoring true negatives that dominate under severe class imbalance. For hybrid attack or multiple attack videos (Table 2 and 3), a prediction is considered correct only if all attack types are simultaneously identified. We also report per-class recall, $TP/(TP + FN)$, to measure correctness conditioned on the presence of the class.

4.2. Numerical Results and discussion

Cross-Scene Cognitive Attack Classification. The results in Table 1 demonstrate that *CADAR* achieves the highest overall accuracy (74.6%) and F1-score (84.2%) and its average accuracy exceeding the next best baseline models (GPT-5-mini) by 16%. *CADAR* attains the highest class-specific accuracy and F1 in 23 out of 25 attack-scene blocks. The margin is especially large for Visual and Obstruction attacks, where *CADAR* delivers strong gains, underscoring its robustness to fine-grained semantic distortions. We also observe that VLM baselines exhibit substantial accuracy fluctuations across different scenes, indicating

Table 1. Attack-wise Accuracy and F1-score (%) across scenes for frame-level and video-level baselines. Each attack type forms a block, with sub-rows indicating methods. Columns represent scene categories. CADAR outperforms VLM-based (GPT-5-mini, Gemini-2.5-flash) and supervised video models (3D-ResNet, ViViT) in 23 out of 25 attack–scene blocks.

			Class-specific Accuracy / F1-score (%)					
Attack	Detection	Method	School&Office	Traffic&Mobility	Industrial&Infra	Medical&Lab	Public&Home	Avg Acc/F1
Text	Frame	CADAR (Ours)	68.8 / 80.2	69.0 / 78.2	78.5 / 91.1	71.0 / 82.4	72.7 / 80.1	72.0 / 82.4
		GPT-5-mini	50.6 / 67.2	59.6 / 74.7	52.1 / 68.5	52.5 / 68.9	61.1 / 75.9	55.2 / 71.0
		Gemini-2.5-flash	73.0 / 84.4	58.4 / 73.7	77.4 / 87.2	41.6 / 58.8	51.4 / 67.9	60.4 / 74.4
	Video	3D ResNet ViViT	12.5 / 22.2 26.3 / 41.7	10.9 / 19.6 70.6 / 82.8	12.8 / 22.6 25.0 / 42.1	15.6 / 27.0 33.3 / 50.0	40.0 / 57.1 43.5 / 60.6	18.4 / 29.7 39.7 / 55.4
Visual	Frame	CADAR (Ours)	80.3 / 92.1	79.2 / 85.2	67.2 / 78.9	69.7 / 81.0	70.4 / 84.5	73.4 / 84.3
		GPT-5-mini	51.0 / 67.6	61.5 / 76.2	50.6 / 67.2	45.5 / 62.5	59.8 / 74.9	53.7 / 69.7
		Gemini-2.5-flash	49.5 / 66.3	76.7 / 86.8	43.9 / 61.0	58.7 / 74.0	40.8 / 58.0	53.9 / 69.2
	Video	3D ResNet ViViT	16.5 / 28.3 22.2 / 36.4	7.2 / 13.3 44.4 / 61.5	8.3 / 15.3 20.0 / 33.3	12.7 / 22.5 28.6 / 44.4	8.3 / 15.4 25.0 / 40.0	10.6 / 19.0 28.0 / 43.1
Obstruction	Frame	CADAR (Ours)	78.2 / 87.3	85.1 / 91.9	81.2 / 89.3	78.3 / 89.1	80.7 / 92.1	80.7 / 89.9
		GPT-5-mini	75.4 / 85.9	46.0 / 63.0	61.7 / 76.3	46.3 / 63.3	63.2 / 77.4	58.5 / 73.2
		Gemini-2.5-flash	64.7 / 78.6	51.6 / 68.1	55.0 / 71.0	36.9 / 53.9	43.6 / 60.7	50.4 / 66.5
	Video	3D ResNet ViViT	8.1 / 15.0 50.0 / 66.7	3.0 / 5.9 54.5 / 70.6	5.0 / 9.5 46.2 / 63.2	9.4 / 17.2 71.4 / 83.3	4.5 / 8.7 41.7 / 42.3	6.0 / 11.3 52.8 / 65.2
Injection	Frame	CADAR (Ours)	76.9 / 81.1	81.4 / 89.5	72.2 / 78.9	73.5 / 79.0	74.0 / 82.6	75.6 / 82.2
		GPT-5-mini	69.1 / 81.7	60.7 / 75.6	56.3 / 72.0	55.2 / 71.2	63.3 / 77.5	60.9 / 75.6
		Gemini-2.5-flash	70.1 / 82.4	54.1 / 70.2	62.8 / 77.2	61.5 / 76.1	53.1 / 69.4	60.3 / 75.0
	Video	3D ResNet ViViT	6.6 / 12.3 30.0 / 46.2	12.8 / 22.7 45.5 / 62.5	9.8 / 17.8 20.0 / 33.3	13.3 / 23.4 25.0 / 40.0	15.7 / 27.2 55.6 / 71.4	11.6 / 20.7 35.2 / 50.7
Non attack	Frame	CADAR (Ours)	75.3 / 80.8	68.9 / 80.2	71.6 / 84.9	74.6 / 85.2	67.2 / 79.4	71.5 / 82.1
		GPT-5-mini	71.8 / 83.6	52.1 / 68.5	62.8 / 77.1	70.5 / 82.7	68.1 / 81.0	65.1 / 77.4
		Gemini-2.5-flash	54.2 / 70.3	38.2 / 55.3	44.0 / 61.1	41.1 / 58.3	44.1 / 61.2	44.3 / 61.2
	Video	3D ResNet ViViT	8.5 / 15.7 40.0 / 57.1	11.7 / 20.9 15.4 / 26.7	8.6 / 15.9 7.7 / 14.3	15.9 / 27.5 40.0 / 57.1	29.4 / 45.5 30.8 / 47.1	14.8 / 25.1 26.8 / 40.5
Overall Mean (per method):		CADAR: 74.6 / 84.2	GPT-5-mini: 58.7 / 73.4	Gemini-2.5-flash: 53.9 / 69.3	3D ResNet: 12.3 / 21.1	ViViT: 36.5 / 51.0		

Table 2. Recall comparison for hybrid-attack scenarios(%), CADAR achieves the highest recall across both hybrid combinations.

Hybrid Attack Type	Recall of Method (%)			
	GPT-5-mini	Grok-4-fast	CADAR	Gemini-2.5-flash
Text+Visual	35.0	41.2	76.4	29.9
Obstruction+Injection	32.7	26.1	80.4	45.8

Table 3. Recall (%) of attack detection under different numbers of simultaneous attacks. CADAR maintains strong performance as the number of attacks increases, whereas baseline models degrade substantially.

Method	Number of Attacks				Avg
	1 Attack	2 Attacks	3 Attacks	4 Attacks	
GPT5-mini	66.4	40.2	31.5	29.6	41.9
Grok-4-fast	69.7	55.0	51.3	36.5	53.1
Gemini-2.5-flash	61.7	43.6	33.0	31.5	42.5
CADAR	81.0	78.3	75.2	67.2	75.4

that their reasoning processes are unstable and prone to hallucination. Because they rely on jointly processing all textual and visual inputs, they become increasingly sensitive to misleading cues, malicious prompts, and complex visual environments. This vulnerability highlights the advantage of our neuro-symbolic approach, which offers more controlled and explainable reasoning under diverse AR scenarios.

On the other hand, traditional video-classification models (3D-ResNet and ViViT) perform poorly across all attack types, suggesting that the models are inadequate for capturing

the fine-grained semantic inconsistencies central to cognitive attack detection. Their limited performance is further exacerbated by the lack of large-scale, AR-specific training data, making these models particularly weak in zero- or few-shot scenarios.

Overall, the results indicate that *CADAR*’s neurosymbolic design and temporal reasoning components provide strong advantages for detecting subtle, localized, and multi-instance AR cognitive attacks, achieving robust performance across diverse environments and attack conditions.

Hybrid and Multiple Attacks Table 2 and 3 evaluate model robustness in hybrid and multi-attack settings. In hybrid-attack scenarios, *CADAR* attains the highest recall for both Text+Visual(76.4%) and Obstruction+Injection(80.4%) cases and surpasses all VLM baselines by a substantial margin. This demonstrates that the proposed neurosymbolic graph reasoning framework can effectively detect the attacks separately. In multiple attack scenarios, the performance of VLM baselines declines sharply as attack complexity grows, whereas *CADAR* retains strong recall, from 81.0% for a single attack to 67.2% for four simultaneous attacks, showing the smallest performance drop among all compared methods. These results highlight *CADAR*’s resilience in multi-attack scenarios and its clear advantage in detecting objects in parallel.

Effect of Scene Complexity. The recall trends in Fig. 7(a) reveal a clear divergence between VLM-based baselines

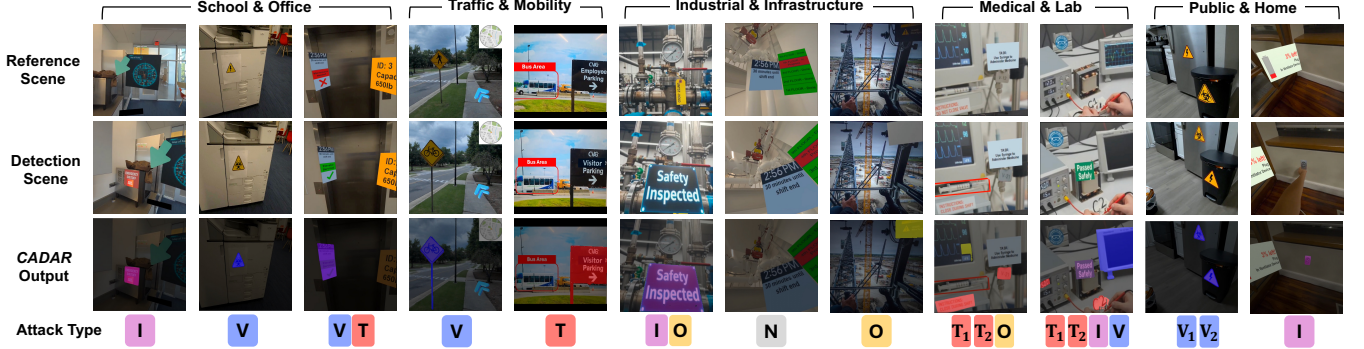


Figure 6. Visualization frame examples of *CADAR* outputs across five scene categories in *CADAR*-50K 375 videos. Rows show the Reference Scene, Detection Scene, and *CADAR* Output. Output attack types: I = Injection, V = Visual, T = Text, O = Obstruction, N = Non-attack. Some examples contain hybrid or multiple attacks.

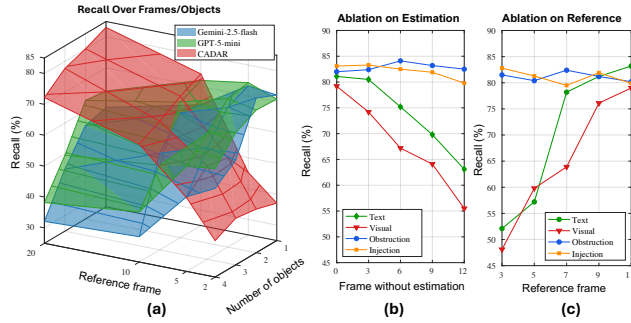


Figure 7. Recall under increasing reference frames and object quantities (a), and ablation studies on the estimation module (b) and the number of reference frames (c).

and *CADAR* as scene complexity increases. For GPT-5-mini and Gemini-2.5-flash, recall drops sharply when the number of key objects and frames increases, suggesting that conventional VLMs struggle with reasoning over long visual and textual context and are easily affected by irrelevant or distracting content. In contrast, *CADAR* performance remains stable as reference frames increase, since it extracts complex scene information into a structural spatial-temporal perception graph. *CADAR* improves its decision accuracy by accumulating prior evidence through additional reference frames, and evaluates objects and attacks separately rather than relying on a single holistic reasoning step. The *CADAR* design results in substantially smaller performance degradation compared with standard VLMs.

4.3. Ablation Study

Estimation Module. Fig. 7(b) shows our ablation study examining the impact of the estimation module in particle filtering. When a polluted node that might have bad data due to VLM unstable output or due to an undetected attack, the estimation module’s weighting and resampling can effectively prevent it from corrupting the ground truth knowledge. We randomly sample 6 videos per base attack type (24 in total) from the test split for the ablation study. As

shown in the left plot of the figure, for text and visual modification attacks, the recall drops significantly as the number of frames without the estimation module increases. Specifically, the recall for text attacks decreases from approximately 81% to 63% and from approximately 79% to 55% for visual attacks. This indicates that the estimation module is crucial for correctly identifying and mitigating errors in more subtle attack types. In contrast, for obstruction and injection attacks, the recall remains relatively stable, suggesting these attacks are more straightforward to detect and less dependent on the estimation module’s corrective capabilities.

Reference Frames. Fig. 7(c) shows the impact of the number of reference frames on recall. A larger number of reference frames, or more prior knowledge of the scene, will result in higher recall. For text and visual modification attacks, increasing the number of reference frames from 3 to 11 significantly improves performance. The recall for text attacks increases from around 52% to 83% and from around 48% to 79% for visual attacks. This demonstrates that a larger set of reference frames provides more prior knowledge for more accurate statistical reasoning, which is essential for detecting subtle text and visual changes. Similar to the estimation module study, the recall for obstruction and injection attacks remains largely unaffected by the number of reference frames, consistently staying around 80%. This reinforces the finding that these attacks are less sensitive to the amount of contextual information and prior knowledge.

5. Conclusion

We presented *CADAR*, a neuro-symbolic framework that couples a perception graph with particle-filter reasoning to yield interpretable attack detection in AR/MR. On the *CADAR*-50K dataset, *CADAR* performs strongly across all attack types, and ablations highlight the value of our estimation module and richer reference histories. Overall, *CADAR* highlights the value of neuro-symbolic design by combin-

ing the adaptability of foundation models with symbolic reasoning for a robust and interpretable AR safety mechanism.

References

- [1] Manoj Acharya, Anirban Roy, Kaushik Koneripalli, Susmit Jha, Christopher Kanan, and Ajay Divakaran. Detecting out-of-context objects using graph context reasoning network. In *IJCAI*, 2022. 1
- [2] Anurag Arnab, Mostafa Dehghani, Georg Heigold, Chen Sun, Mario Lučić, and Cordelia Schmid. Vivit: A video vision transformer. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 6836–6846, 2021. 6
- [3] Md Abdul Awal, Md Abu Rumman Refat, Feroza Naznin, and Md Zahidul Islam. A particle filter based visual object tracking: A systematic review of current trends and research challenges. *International Journal of Advanced Computer Science & Applications*, 14(11), 2023. 2
- [4] Bikram Pratim Bhuyan, Amar Ramdane-Cherif, Ravi Tomar, and TP Singh. Neuro-symbolic artificial intelligence: a survey. *Neural Computing and Applications*, 36(21):12809–12844, 2024. 2
- [5] Xiaojun Chang, Pengzhen Ren, Pengfei Xu, Zhihui Li, Xiaojiang Chen, and Alex Hauptmann. A comprehensive survey of scene graphs: Generation and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(1): 1–26, 2021. 2
- [6] S. Y. Chen. Kalman filter for robot vision: A survey. *IEEE Transactions on Industrial Electronics*, 59(11):4409–4420, 2012. 2
- [7] Zhe Chen et al. Bayesian filtering: From kalman filters to particle filters, and beyond. *Statistics*, 182(1):1–69, 2003. 2
- [8] Kaiming Cheng, Jeffery F. Tian, Tadayoshi Kohno, and Franziska Roesner. Exploring user reactions and mental models towards perceptual manipulation attacks in mixed reality. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 911–928, Anaheim, CA, 2023. USENIX Association. 1, 2
- [9] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blstein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025. 6
- [10] Lauren Nicole DeLong, Ramon Fernández Mir, and Jacques D Fleuriot. Neurosymbolic ai for reasoning over knowledge graphs: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 2024. 2
- [11] Huiyu Duan, Xiongkuo Min, Yucheng Zhu, Guangtao Zhai, Xiaokang Yang, and Patrick Le Callet. Confusing image quality assessment: Toward better augmented reality experience. *IEEE Transactions on Image Processing*, 31:7206–7221, 2022. 1
- [12] Zhaopeng Gu, Bingke Zhu, Guibo Zhu, Yingying Chen, Ming Tang, and Jinqiao Wang. Anomalygpt: Detecting industrial anomalies using large vision-language models. In *Proceedings of the AAAI conference on artificial intelligence*, pages 1932–1940, 2024. 1
- [13] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. Knowledge graphs. *ACM Computing Surveys (Csur)*, 54(4):1–37, 2021. 2
- [14] Hans R Künsch. Particle filters. 2013. 2
- [15] Yanjie Li, Bin Xie, Songtao Guo, Yuanyuan Yang, and Bin Xiao. A survey of robustness and safety of 2d and 3d deep learning models against adversarial attacks. *ACM Computing Surveys*, 56(6):1–37, 2024. 1
- [16] Matthias Minderer, Alexey Gritsenko, and Neil Houlsby. Scaling open-vocabulary object detection, 2024. 10
- [17] Furkan Mumcu and Yasin Yilmaz. Multimodal attack detection for action recognition models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2967–2976, 2024. 2
- [18] Uzma Nawaz, Mufti Anees-ur Rahaman, and Zubair Saeed. A review of neuro-symbolic ai integrating reasoning and learning for advanced cognitive systems. *Intelligent Systems with Applications*, page 200541, 2025. 2
- [19] Shilin Qiu, Qihe Liu, Shijie Zhou, and Chunjiang Wu. Review of artificial intelligence adversarial attack and defense technologies. *Applied Sciences*, 9(5):909, 2019. 1
- [20] Chenfan Qu, Chongyu Liu, Yuliang Liu, Xinhong Chen, Dezhi Peng, Fengjun Guo, and Lianwen Jin. Towards robust tampered text detection in document image: New dataset and new solution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5937–5946, 2023. 1
- [21] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. Sam 2: Segment anything in images and videos, 2024. 10
- [22] Anirban Roy, Adam Cobb, Ramneet Kaur, Sumit Jha, Nathaniel D Bastian, Alexander Berenbeim, Robert Thomson, Iain Cruickshank, Alvaro Velasquez, and Susmit Jha. Zero-shot detection of out-of-context objects using foundation models. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 9186–9195. IEEE, 2025. 1
- [23] Dhruv Shah, Blazej Osinski, Brian Ichter, and Sergey Levine. Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action, 2022. 2
- [24] Ali Teymourian, Andrew M Webb, Taha Gharaibeh, Arushi Ghildiyal, and Ibrahim Baggili. Sok: Come together—unifying security, information theory, and cognition for a mixed reality deception attack ontology & analysis framework. *arXiv preprint arXiv:2502.09763*, 2025. 1
- [25] Du Tran, Heng Wang, Lorenzo Torresani, Jamie Ray, Yann LeCun, and Manohar Paluri. A closer look at spatiotemporal convolutions for action recognition. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 6450–6459, 2018. 6

- [26] Junke Wang, Zuxuan Wu, Jingjing Chen, Xintong Han, Abhinav Shrivastava, Ser-Nam Lim, and Yu-Gang Jiang. Objectformer for image manipulation detection and localization. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2364–2373, 2022. 1
- [27] Yanming Xiu and Maria Gorlatova. Detecting visual information manipulation attacks in augmented reality: A multimodal semantic reasoning approach. *IEEE Transactions on Visualization and Computer Graphics*, 31(11):9645–9655, 2025. 6
- [28] Yanming Xiu and Maria Gorlatova. Detecting visual information manipulation attacks in augmented reality: A multimodal semantic reasoning approach, 2025. 1, 2
- [29] Yanming Xiu, Tim Scargill, and Maria Gorlatova. Viddar: Vision language model-based task-detrimental content detection for augmented reality. *IEEE Transactions on Visualization and Computer Graphics*, 31(5):3194–3203, 2025. 1
- [30] Yanming Xiu, Tim Scargill, and Maria Gorlatova. Viddar: Vision language model-based task-detrimental content detection for augmented reality. *IEEE transactions on visualization and computer graphics*, 2025. 1
- [31] Sanggeon Yun, Ryoza Masukawa, Minhyoung Na, and Mohsen Imani. Missionggnn: Hierarchical multimodal gnn-based weakly supervised video anomaly recognition with mission-specific knowledge graph generation. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 4736–4745. IEEE, 2025. 2
- [32] Chiyu Zhang, Lu Zhou, Xiaogang Xu, Jiafei Wu, and Zhe Liu. Adversarial attacks of vision tasks in the past 10 years: A survey. *ACM Computing Surveys*, 2024. 2
- [33] Guoquan Zheng and Liang Yuan. A review of qoe research progress in metaverse. *Displays*, 77:102389, 2023. 1

Appendix

A. CADAR Implementation Details

This section provides the full implementation details of CADAR, covering perception-graph construction, multimodal embedding generation, temporal reasoning, and statistical attack detection. The system follows a modular architecture implemented across the `Node.py`, `Edge.py`, `Graph.py`, `NodeSet.py`, `EdgeSet.py`, and `Utils.py` modules in our codebase.

A.1. Perception Graph Construction

Object Detection and Segmentation. For each frame, CADAR extracts object proposals using two vision modules:

- Zero-shot object detection: `google/owlv2-base-patch16-ensemble` [16], which produces category-agnostic bounding boxes and confidence scores.
- Segmentation masks: SAM2 [21], used to obtain object-level silhouettes for robust visual embedding extraction.

Textual Attribute Generation. To obtain semantic descriptions associated with the detected region, we use a lightweight VLM (GPT-4.1) generates short, structured textual descriptions. These descriptions include attributes such as object type, color, role, contextual relevance, and scene relationships. The attributes also include numerical importance, reasonability and probability.

Multimodal Embedding Extraction. Each detected object is represented by three embedding modalities, each extracted using a specialized encoder:

- Name embedding: `paraphrase-MiniLM-L6-v2` (384-d).
- Description embedding: `all-mpnet-base-v2` (768-d).
- Visual embedding: a ResNet-34 feature encoder applied to SAM2-extracted object crops (512-d).

These multimodal perceptual vectors are then projected using PCA for statistical reasoning (described below).

Node and Edge Formation.

Implementation Details. The software architecture is structured around two primary classes, `Node.py` and `Edge.py`, which manage the lifecycle and serialization of the perception graph components. The `Node` class orchestrates the bottom-up extraction pipeline, initializing the vision models to generate object masks and invoking the VLM to populate attribute fields (e.g., reasonability, importance) while computing and storing dense multimodal embeddings. Complementing this, the `Edge` class executes the relational analysis by ingesting the instantiated node list and source image; it prompts the LLM to infer pairwise dependencies (such as spatial or functional links) and encodes these descriptions into vector format, thereby establishing the graph’s connectivity before serializing the complete state to compressed binary files.

A.2. Graph Backbone and Temporal Tracking

Graph Management. The `Graph.py` module serves as the fundamental container for encapsulating the state of the perception system at discrete timestamps. It aggregates detected entities and their pairwise relationships into a coherent network structure, storing lists of `Node` and `Edge` instances alongside temporal metadata and references to the source imagery. Beyond acting as a static data structure, this module facilitates the serialization of the complete graph state into compressed binary formats to ensure efficient persistence and provides visualization utilities that render the probabilistic attributes and topological connectivity of the scene using network layout algorithms.

NodeSet and EdgeSet Evolution. To transform isolated frame-level detections into continuous historical tracks, the system relies on the `NodeSet.py` and `EdgeSet.py` modules. The `NodeSet` class maintains the identity of objects over time by associating new detections with existing tracks via multi-modal embedding similarity (visual, semantic, and nominal) rather than simple spatial overlap. It implements a particle filter-based mechanism to estimate stable node attributes from noisy observations and employs statistical methods—such as PCA and Mahalanobis distance—to detect anomalies or adversarial attacks. Complementing this, the `EdgeSet` tracks the evolution of semantic and spatial relationships between these persistent entities, preserving the history of interactions to support high-level reasoning as the scene evolves.

A.3. Statistical Anomaly Detection

Dimensionality Reduction and Feature Whitening. To ensure robust distance metrics in high-dimensional spaces, we apply Principal Component Analysis (PCA) to the embedding history of each track. For a set of historical embedding vectors $\mathbf{X} \in \mathbb{R}^{N \times D}$, we compute a projection that preserves a specific variance ratio. Specifically, for node textual and visual embeddings, we retain 99% of the variance ($v = 0.99$), while for edge description embeddings, we retain 90% ($v = 0.90$). For visual embeddings, we additionally apply whitening to normalize the principal components to unit variance. This projection results in a dynamic latent dimension k (typically $16 \leq k \leq 32$), which defines the degrees of freedom for subsequent statistical tests.

Regularized Mahalanobis Distance. We model the distribution of "clean" reference frames for any given object or relationship as a multivariate Gaussian. To assess whether a new observation x_t belongs to this distribution, we calculate the squared Mahalanobis distance. To prevent numerical instability when the covariance matrix is singular (common in short history windows), we introduce a regularization term $\epsilon \mathbf{I}$:

$$d^2(x_t) = (x_t - \mu)^\top (\Sigma + \epsilon \mathbf{I})^{-1} (x_t - \mu) \quad (22)$$

where μ is the mean of the historical embeddings, Σ is the covariance matrix, and $\epsilon = 10^{-6}$ is a smoothing factor.

Chi-Square Hypothesis Testing. Anomalies are identified by comparing the squared distance against a critical value derived from the Chi-square distribution with k degrees of freedom. An observation is flagged as an attack (e.g., semantic modification or visual spoofing) if:

$$d^2(x_t) > \chi_{1-\alpha, k}^2 \quad (23)$$

where α represents the significance level. We employ adaptive sensitivity thresholds: $\alpha = 0.10$ for textual deviations, $\alpha = 0.05$ for visual deviations, and $\alpha = 0.01$ for

edge relationship deviations. This statistical framework allows *CADAR* to detect subtle adversarial manipulations that deviate from the established semantic baseline.

A.4. Particle-Filter-Based Temporal Reasoning

To mitigate the stochasticity of VLM outputs and detection noise, we implement a particle filter that maintains a belief state over node attributes. Unlike standard tracking which estimates position, our filter estimates semantic consistency.

Weight Update. For a node n with history H , we define the weight w_i of the i -th particle (attribute hypothesis) based on its similarity to the incoming observation z_t . For scalar attributes (probability, importance, reasonability), weights follow a Gaussian kernel:

$$w_i \propto \exp\left(-\frac{(v_i - z_t)^2}{2\sigma^2}\right) \quad (24)$$

For high-dimensional embedding attributes, weights are derived from cosine similarity, scaled by a confidence factor λ :

$$w_i \propto \exp\left(\lambda \cdot \frac{\mathbf{e}_i \cdot \mathbf{z}_t}{\|\mathbf{e}_i\| \|\mathbf{z}_t\|}\right) \quad (25)$$

where $\lambda = 1$ is the scaling factor used to tune the confidence in the top matches.

Resampling and Estimation. We perform importance resampling to filter outlier descriptions generated by the VLM. The final estimated state $\hat{x}_t$ is computed as the weighted centroid of the resampled population, effectively smoothing the semantic trajectory of the object and rejecting transient classification errors.

A.5. Supporting Utilities

Utility and Pre-processing. The `utils.py` module serves as a comprehensive toolkit for low-level visual processing and mathematical operations underpinning the higher-level graph logic.

A.6. Summary

Overall, *CADAR* integrates zero-shot perception, multi-modal symbolic representation, PCA-based statistical modeling, and particle-filter temporal reasoning into a unified framework for robust AR cognitive attack detection. The modular architecture allows flexible substitution of detectors, encoders, and reasoning mechanisms, while ensuring reproducibility across all experimental settings.

B. CADAR-50K Dataset Samples

Representative sample videos for each class (excluding the No Attack category) is shown below. The full dataset consists of 375 videos across diverse scenes.

Table 4. Hyper-parameters for Statistical Detection and Tracking.

Parameter	Value	Description
Matching Threshold	0.2	Similarity floor for node association
Min. History Window	5 frames	Minimum samples required for PCA/Mahalanobis
PCA Variance (Nodes)	0.99	Variance retained for text/image embeddings
PCA Variance (Edges)	0.90	Variance retained for edge descriptions
Regularization (ϵ)	10^{-6}	Covariance matrix stabilization term
α (Text)	0.10	Significance level for node text anomalies
α (Image)	0.05	Significance level for visual appearance anomalies
α (Edge)	0.01	Significance level for relationship anomalies
Particle Scale (λ)	1.0	Exponential scaling for embedding similarity weights

Figure 8. Sample dataset content for each AR Attack Class. For video examples, please refer to **CADAR_demo.mp4** in the supplementary materials.

B.1. Dataset Construction

For the real-world scenes in the CADAR-50K dataset, we collected additional samples using a Meta Quest 3 headset and a cellphone camera. The video dataset contains both malicious cognitive attacks and benign augmented reality content. All data was recorded using the built-in pass-through camera of a Meta Quest headset and a Samsung Galaxy S24. We employed two distinct data generation

pipelines to ensure diversity in environmental contexts and lighting conditions.

B.2. In-Situ Augmented Reality Overlay

The first method involved augmenting real-world environments with manipulated overlays to simulate attacks in situ. We first identified target objects within a physical scene (e.g., a store sign displaying a discount). A high-resolution photograph of the target was captured and subjected to digi-

tal manipulation. For instance, altering text from "50% off" to "150% off" to simulate a text manipulation attack.

The manipulated asset was imported into the Unity engine and mapped onto a planar mesh (Quad). We utilized Unity’s scaling and transformation tools to align the virtual overlay with the physical object’s dimensions and perspective in the real world. To introduce benign confounders, we analyze the scene context and suggest appropriate non-malicious objects. Based on these suggestions, synthetic 3D or 2D assets were generated and anchored within the scene. During the recording phase, a controller input triggered the visibility of the overlay, allowing us to capture the transition between the benign state and the attack state dynamically.

B.3. Synthetic Scene Projection

To expand the dataset beyond accessible physical locations, we utilized a fully synthetic generation pipeline. In the Unity environment, we instantiated a large-scale Quad (approximately $5m \times 5m$) to serve as a dynamic backdrop.

We utilized a generative image model to synthesize realistic background scenes. The generation of scenes containing specific target objects is required for occlusion or manipulation attacks. Once the generated scene was applied to the backdrop, attack artifacts were positioned in the foreground. Although the background was two-dimensional, we achieved a pseudo-three-dimensional effect by strategically angling the foreground objects and the camera perspective. Similar to the in-situ method, the attack artifacts were toggled via controller input during recording to capture paired data samples.



Figure 9. (a) First-person view from the AR headset: the virtual object (e.g. a dog and textual cues) is interactive and can be toggled on or off using the “Button B” on the right controller, while “Button A” starts or stops video recording. (b) Third-person view showing the data collector wearing the Meta Quest 3 headset and interacting with the AR environment using handheld controllers.

C. Video Baseline Implementation Details

We provide the full implementation details of the supervised video baselines used in our experiments: the 3D-ResNet model [?] and the transformer-based ViViT model [?]. Both baselines follow standard practice in video classification and are trained under consistent data splits, preprocessing steps, and evaluation metrics. The descriptions be-

low summarize the implementation in our released training code.

C.1. Dataset Splits and Sampling

All video models use the same dataset partitions:

- Train / Validation / Test: 60% / 20% / 20%, stratified by attack type and scene category.
- One clip is extracted per video for all splits, matching the single-view evaluation protocol.

To mitigate class imbalance during training, we adopt a class-balanced sampling strategy. Let c_i be the number of training videos for class i ; class weights are:

$$w_i = \frac{1}{c_i + 10^{-6}}, \quad p_i = \frac{w_i}{\sum_j w_j}, \quad (26)$$

and training videos are sampled according to p_i . This sampling procedure matches the implementation in our 3D-ResNet code.

C.2. Clip Construction and Preprocessing

For both ViViT and 3D-ResNet, AR videos are processed into short clips using a uniform temporal sampling pipeline:

- 20-frame clips: obtained using `UniformTemporalSubsample(20)`.
- Spatial resizing: frames are resized to 400×400 .
- Pixel normalization: mean $[0.45, 0.45, 0.45]$, std $[0.225, 0.225, 0.225]$.

Frames are arranged as tensors of shape (T, C, H, W) with $T = 20$. No additional augmentations (cropping, flipping, jittering) are applied, avoiding synthetic distortions that could alter AR semantics.

C.3. 3D-ResNet Baseline

Our 3D CNN baseline is implemented using the `r3d_l18` architecture from `torchvision` (18-layer 3D-ResNet). We detail the full setup below.

Architecture.

- Model: R3D-18 (3D ResNet).
- Pretraining: Kinetics-400 weights from Torchvision.
- Final classifier layer replaced with a 5-way linear head.

Loss Function. We employ a Focal Loss with $\gamma = 2.0$:

$$\mathcal{L}_{\text{focal}} = (1 - \exp(-\text{CE}(\mathbf{x}, y)))^\gamma \text{CE}(\mathbf{x}, y), \quad (27)$$

which improves stability under severe class imbalance.

Optimization.

- Optimizer: AdamW
- Learning rate: 5×10^{-5}
- Weight decay: 5×10^{-3}

- Scheduler: StepLR (step=10, $\gamma=0.1$)
- Batch size: 20
- Epochs: 20

Precision Training. Mixed-precision (FP16) training is performed using HuggingFace Accelerate with `autocast()` enabled, matching our code. All layers are trainable; no freezing is applied.

C.4. ViViT Baseline

We adopt the ViViT-B/16x2 (Factorised Encoder) model as our transformer baseline:

- Patch size: 16×16
- Hidden dimension: 768
- Transformer layers: 12
- Heads: 12
- Pretraining: Kinetics-400

The model receives the same 20-frame clips and performs temporal factorization before spatial tokenization as in the original ViViT design.

Training. We train ViViT for 20 epochs using:

- Batch size: 16
- AdamW optimizer ($1e^{-4}$ learning rate, $5e^{-3}$ weight decay)
- Cosine learning-rate decay
- Warmup steps: 10% total steps
- Mixed precision (FP16) using Accelerate

C.5. Evaluation Metrics

To ensure consistency with the metrics defined in the main paper, we evaluate both video models using the following measures:

Class-Specific Accuracy. For each class k :

$$\text{Acc}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k + \text{FP}_k}.$$

Precision, Recall, F1. Computed using scikit-learn with `zero_division=0`.

Confusion Matrix. Collected over the test split and used to compute TP/FP/FN counts for each class.

C.6. Fairness and Consistency

To ensure a fair comparison with CADAR and VLM baselines:

- All models receive the same raw input videos.
- No prompts, symbolic structures, or additional annotations are provided to video baselines.
- Clip preprocessing (20 frames, resizing, normalization) is identical across all supervised baselines.

Overall, these settings yield strong and transparent supervised video baselines, allowing us to isolate the contributions of CADAR’s neurosymbolic reasoning.

C.7. Model Architectures and Training

For our supervised-learning baselines—3D ResNet and ViViT. Training hyper-parameters are shown below. We conducted all training and evaluation using Google Colab with NVIDIA A100 GPUs (40GB memory). The experiments were run in a Linux environment via Colab’s remote infrastructure, conducted using PyTorch 2.6.0 with Python 3.11.

	3D ResNet			ViViT		
	Value	Range	# Tried	Value	Range	# Tried
initial/end lr	5e-5	[5e-5, 5e-3]	5	1e-4	[5e-5, 5e-3]	4
weight decay	5e-3	[1e-4, 1e-1]	5	5e-3	[1e-3, 1e-1]	4
batch size	20	[8–20]	3	16	[4–16]	2
epochs	20	[5–50]	6	20	[5–50]	4
scheduler	StepLR	–	–	Cosine	–	–
optimizer	AdamW	–	–	AdamW	–	–

Table 5. Training hyper-parameters, tested ranges, and number of values tried for ViViT and 3D ResNet.

We use ViViT-B/16x2 (pretrained on Kinetics-400) as the backbone for our ViViT model, and a 3D ResNet-18 for the ResNet baseline. Both models are trained on the full training set using inverse-frequency sampling to balance class distributions. The classification task involves five classes: the four attack types, along with an additional No Attack category. All input videos are resized to 400×400 pixels and normalized using a mean of $[0.45, 0.45, 0.45]$ and a standard deviation of $[0.225, 0.225, 0.225]$. Each input consists of 20 uniformly sampled frames from a video clip. We use Focal Loss with a gamma of 2.0 as the training objective. Mixed precision (FP16) training is enabled via HuggingFace Accelerate for both models. ViViT is trained with a cosine learning rate scheduler, while the 3D ResNet uses a StepLR scheduler (step size = 10, gamma = 0.1). Final hyperparameter configurations for both models were selected based on the settings that yielded the highest validation accuracy.

D. Vision–Language Model (VLM) Baseline Implementation Details

We evaluate three state-of-the-art multimodal VLMs as baselines: GPT-5-mini (OpenAI), Gemini-2.5-flash (Google), and Grok-4-fast (xAI). All three models support multi-image inputs and natural language reasoning, enabling them to perform frame-level attack classification without any training or fine-tuning. This section details our prompt design, image preprocessing strategy, and inference procedure, following our released implementation.

D.1. Input Frames and Preprocessing

Each AR video is converted into multiple reference and attack frames using the procedure in our VLM baseline code. We extract:

- **N reference frames** uniformly from the first 30–40% of the video, and
- **M attack frames** uniformly from the last 25%.

Images are resized to a maximum resolution of 800×800 while preserving aspect ratio. For GPT-5 and Grok, images are encoded as JPEG base64 strings. Gemini receives PIL image objects directly, as required by the `generate_content` interface.

D.2. Unified Prompt for All VLM Baselines

To ensure fair comparison across models, all VLMs use the exact same prompt (text only; no model-specific tuning). We introduce the four base attack types used in CADAR—text modification, visual modification, obstruction, and injection—together with detailed natural-language definitions. The full prompt is shown below.

You are an AR video attack analyst. Your job is to identify the presence of 4 different kinds of AR attacks and non-attacks in the video with the scene context. The definitions for each attack are as follows:

A text modification attack manipulates textual information embedded in the environment, thereby changing the semantic message conveyed by the scene. For example, replacing a “NO PARKING FIRE LANE” sign with “FREE PARKING FIRE LANE” directly reverses the intended directive and can create safety hazards. Such attacks are consequential when text provides primary contextual or navigational cues.

A visual modification attack alters an object’s appearance—its color, shape, or iconography—or repositions it implausibly. Examples include transforming a pedestrian-crossing sign into a bicycle-crossing sign, switching a green traffic signal to red, or relocating a stop sign to the center of an intersection. These manipulations can be quantified by analysing pixel-level discrepancies and deviations from expected spatial distributions.

An obstruction attack hides or deletes critical objects, thereby eliminating essential information and distorting scene semantics. For example, erasing an emergency-exit sign in a fire evacuation scenario removes a safety-critical cue and disrupts the relational structure of the environment.

An Injection attack introduces new, contextually inconsistent, or misleading objects that distort scene semantics. A fake detour sign placed on a clear street, or fictitious warning icons inserted into an AR overlay, may mislead human users and perception systems. Detection relies on identifying unexpected objects that conflict with the scene’s established structure and context.

A non-attack refers to a video in which the AR content remains semantically consistent with the physical environment and does not alter the user’s understanding of the scene.

The following input will contain video frames and context. Please output the class. Note that there may be more than one kind of attack present in the video, or no attack at all. Along with the attack, mention the object the attack was performed on, or briefly describe the attack in the context of the video.

This prompt is prepended to the multimodal input for all VLMs.

D.3. Inference Procedure

Each VLM receives:

- the textual prompt above,
- scene context,
- $N = 20$ reference images, and
- $M = 5$ attack image at a time.

The models are called with their native multimodal APIs:

- GPT-5-mini: OpenAI Chat Completions (image URLs encoded as base64).
- Gemini-2.5-flash: `generate_content` with a mixed list of text + images.
- Grok-4-fast: xAI Chat Completions (OpenAI-compatible JSON format with image URLs).

Each response is required to be valid JSON, following the schema below:

```
[
  {
    "object": "<object>",
    "attack_type": "<classification>",
    "notes": "<short explanation>"
  }
]
```

We apply a lightweight parser that extracts JSON blocks from the raw VLM outputs. Each model produces one prediction per attack frame. This mirrors video-level behavior and reduces variance inherent in single-frame VLM outputs.

D.4. Fairness Measures

To ensure a strict apples-to-apples comparison:

- All VLMs use the same prompt with context, the same set of reference frames, and the same attack frames.
- No model-specific prompt engineering, priming, or hand-crafted reasoning chains are used.
- All text is fed to each VLM exactly once per attack frame; no iterative refinement or self-consistency sampling is applied.

Because VLMs are not adapted to CADAR’s symbolic representation or temporal structure, these baselines reflect the true performance of general-purpose open-world vision–language models on cognitive attack detection.